

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..10

CACHING

 Copy as Markdown

Caching

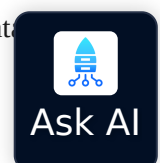
Some work is expensive to repeat: a query that joins several tables, a result assembled from many rows, or a value fetched from another service. Caching stores the result under a key so the next request can read it back instead of doing the work again.

Serverpod's caches are reached from the `Session` object, alongside the database and messaging. Anything you cache must be serializable: the primitives (`int`, `String`, `bool`, `double`, `DateTime`, `Duration`, `ByteData`, `UuidValue`, `Uri`, `BigInt`), collections of them, and your own `models`. Use a model rather than a hand-written class: one with a `toJson` stores fine and throws on read, one without fails immediately with `JsonUnsupportedObjectError`.

The caches

Cache	Where it lives	Use it for
<code>session.caches.local</code>	Memory of the server handling this call	The default choice for cached values.
<code>session.caches.localPrio</code>	Memory of the same server, kept separate	Values you want to keep hot, held apart from ordinary entries.
<code>session.caches.global</code>	Redis, shared by every server instance	Values that must be the same on every instance.

There is a fourth cache, `session.caches.query`, which Serverpod manages for cacheable database queries. It is not a place to put your own objects.



Each local cache holds at most 10,000 entries. When a cache is full, the oldest entry is dropped to make room, based on when it was inserted rather than when it was last read, so a frequently read key can still be evicted.

Cache an object

Read a value with `get`, and write one with `put`:

```
Future<UserData?> getUserData(Session session, int userId) async {
    var cacheKey = 'UserData-$userId';

    // Try the cache first.
    var userData = await session.caches.local.get<UserData>(cacheKey);
    if (userData != null) return userData;

    // Not cached, so load it and store it for five minutes.
    userData = await UserData.db.findById(session, userId);
    if (userData != null) {
        await session.caches.local.put(
            cacheKey,
            userData,
            lifetime: const Duration(minutes: 5),
        );
    }

    return userData;
}
```

The `lifetime` sets how long the entry stays valid. After it elapses, the next read misses and returns `null`. Omit `lifetime` and the entry never expires on its own, so it stays until you invalidate it or it is evicted to make room.

Cache `DateTime` values as UTC, since that is how they come back:

```
await session.caches.local.put('lastUpdate', DateTime.now().toUtc());

var cached = await session.caches.local.get<DateTime>('lastUpdate');
```

Collections work the same way. Give `get` the full type, including what the collection holds:

```
await session.caches.local.put('users', [alice, bob]);

var users = await session.caches.local.get<List<UserData>>('users');
```

Serverpod generates the code to read a collection type back only when your project uses it, such as in an endpoint signature or a model field. Caching a shape that appears nowhere else in your project throws on read.

Load on a miss

The `CacheMissHandler` class folds "read it, and compute it if it is missing" into one call. Serverpod stores whatever the handler returns:

```
Future<UserData?> getUserData(Session session, int userId) async {  
  return session.caches.local.get(  
    'UserData-$userId',  
    CacheMissHandler(  
      () async => UserData.db.findById(session, userId),  
      lifetime: const Duration(minutes: 5),  
    ),  
  );  
}
```

The call returns `null` when the handler does, and nothing is stored in that case. If several requests miss the same key at once, the local caches run the handler once and let the others wait for that result. The Redis-backed global cache runs it per request.

Invalidate entries

A cached value goes stale as soon as the underlying data changes, so remove it when you write:

```
await UserData.db.updateRow(session, userData);  
await session.caches.local.invalidateKey('UserData-${userData.id}');
```

To drop several related entries at once, put them in a group and invalidate the group:

```
await session.caches.local.put(  
  'UserData-$userId',  
  userData,  
  group: 'user-$userId',  
);  
  
// Later, remove every entry in the group.  
await session.caches.local.invalidateGroup('user-$userId');
```

Use `containsKey` to check for an entry without reading it, and `clear` to empty the cache.

⚠ WARNING

Groups are a local-cache feature. On `session.caches.global`, passing `group:` to `put` or calling `invalidateGroup` throws `UnimplementedError`, so code that groups entries breaks when you move it to the global cache.

The global cache and Redis

The global cache uses Redis, which is what makes it shared between server instances. When Redis is not configured, it falls back to an in-memory cache that belongs to a single server process.

WARNING

The fallback applies in every run mode, production included. It exists so you can run code that uses the global cache without a Redis server during development, not as a substitute for Redis in production. In that state each instance keeps its own copy, nothing is shared between instances, and everything is lost on restart.

The server does not warn you about this at normal logging levels, and the [readiness probe](#) still passes, so confirm your Redis configuration before running more than one instance. If Redis is configured but unreachable, the readiness probe fails, so that case does get caught.

Redis counts as configured when your config has a `redis` section that is enabled. A `redis` section that omits `enabled` is treated as enabled. If the section is present but its password is missing, the server fails to start rather than falling back. See [Redis](#) for how to enable it, set the password, and connect a client.

Serverpod does not manage the size of the global cache or decide what it evicts. Redis does, according to how you have configured it, so set a memory limit and an eviction policy there if you cache a large number of objects globally.

Serverpod Cloud does not provide a managed Redis instance, so a deployed app uses the in-memory fallback unless you connect your own. See the [Redis guide](#) for how to set one up.

Send Redis commands directly

Redis supports operations the cache API does not expose, such as atomic counters and sorted sets. You can borrow the connection Serverpod manages:

```
Future<int?> incrementCounter(Session session, String counterName) async {
  var command = await session.serverpod.redisController?.getConnection();
  if (command == null) return null;

  var result = await command.send_object(['INCR', 'my_app:$counterName']);
  return result is int ? result : null;
}
```

The connection is `null` when Redis is disabled or unreachable, including when the global cache is on its in-memory fallback. Responses come back in the shape Redis defines for each command, so check the result before casting it.

The returned `RedisCommand` is Serverpod's own connection: do not close it, and do not replace `Serverpod.redisController`. Namespace your keys so they cannot collide with Serverpod's own cache entries.

Related

- [Sessions](#): the object the caches hang off.
- [Server events](#): the other Redis-backed feature, for messaging between instances.
- [Redis](#): enabling Redis, the password, and connecting a GUI.